

Lab 06: Reversing an Undocumented Protocol

In this lab, you will dissect an undocumented protocol and, optionally, the compression format, for a multiplayer maze game. You will then write a bot client to masterfully defeat (almost) any opponent.

Purpose

To practice using static and dynamic analysis tools to discover and understand undocumented data formats (file formats, protocols, compression algorithms, etc.). You will write documentation for the protocol and/or provide software artifacts that could help a developer write an application that communicates via that protocol. You will write a custom client to communicate with the server providing a computerized player. Essentially, you will learn to observe, dissect, and reuse an existing, undocumented, communications protocol in a new program.

Procedure

You will need access to the following materials:

- A computer with a working, licensed copy of Linux, macOS, or Windows.
- A suite of reverse-engineering tools likely consisting of at least one static and one dynamic analysis tool.
- A copy of Wireshark or some other network protocol analyzer.
- A copy of this lab's "maze escape" program for your OS.

Rules of the Game

This is a fairly simple multiplayer maze game implemented in a client-server model. The server is a dedicated server and implements the rules of the game. Each player uses a client to communicate with the server and participate in the game.

The goal is to be the first player to reach the exit of the maze. The starting position is the top-left space, and the finish is always the bottom-right space. No two players may occupy the same space at the same time. Thus, at the beginning of the game, higher-numbered players are placed to the right of the lower-numbered players. This may seem to create an unfair advantage, but it is possible for players to jump.

Each player is given 3 jumps. Jumps may be used to pass over a wall and/or move past another player. Jumping to a space outside the maze boundary is not allowed. Jumping where there is neither a wall nor another player is not allowed. Walking through a wall or another player is not allowed. If you attempt an illegal move, you lose that turn. Attempting to jump a fourth time does not cause you to lose your turn.

Issue movement commands by typing one of W, A, S, or D when it is your turn. Lower case values indicate walking. Upper case values indicate jumping. An upper case L or Ctrl-C will leave the game.

Players may enter and leave the game at will. When a player enters or re-enters, the token is placed in the top-left space, or an unoccupied space to its right.

Starting the Game

Start the server from the command line:

```
server [MAZE_WIDTH] [MAZE_HEIGHT] [MAX_PLAYERS] [USE_COMPRESSION]
```

Once the server is running, start each client from the command line

```
client [SERVER_ADDRESS]
```

Note that on Linux systems, you will need to instruct the OS to search for binaries in the appropriate directories. You may either add the game's directory to your PATH environment variable, or precede the call with "./". You will also need to tell the OS where to find the shared libraries. Add the game's directory to your LD_LIBRARY_PATH environment variable. Note that this variable typically has no default value.

Reverse Engineering the Protocol

You may choose environments and tools to your preference. If you have previous reverse engineering experience, I encourage you to use a few tools you are less familiar with. As you explore, use each tool for its strengths and consider its weaknesses. For example, you may use a static analysis tool to identify interesting places to put breakpoints in your dynamic tool. You might also be stepping through code in a debugger and become curious about alternative code paths. A static analysis tool can help you understand better how the program got to where it is, and what conditions might have taken it elsewhere.

Take note of how the program is distributed. You should see four components: two dynamic libraries, and two executables. The portions in the shared libraries are likely used by both the client and the server.

If you are paranoid, you may start with some basic static analysis. Check out what other libraries the components communicate with, see what functions they import and export, and convince yourself that the program interacts only with the console and the network. Otherwise, start simply by running the program and getting a feel for how it works. This may provide some insight and help you form hypotheses about the protocol. At this point, you may continue with either static or dynamic analysis as you see fit.

Documenting the Protocol

Use your notes from the previous phase to produce formal documentation for the protocol. You should include this in a special section or appendix in your written report. At a high level, please describe the general operation of the protocol. At a lower level, please describe the messages and their formats.

You may find it useful to produce a simple library compatible with the protocol (in any reasonable language of your choosing). This will also give you a head start in the following phase. Please include a brief description of each message type. Basically, you should provide enough detail that another developer could use your notes to produce a compatible client without additional reverse engineering.

The Maze Data

The maze is transferred from the server to the client using an optionally compressed data format. Decoding the uncompressed format is required. Decoding the compressed format is extra credit. For extra credit, the report should contain a high-level description of how the compression works; the details of the compression should be documented in your code.

As an alternative for figuring out the compression, for extra credit, you may use the API of the provided artifacts to perform the decompression for you. Past experience shows this method may be just as difficult, but you can choose the one that fits your group's expertise.

Forward or Re-Engineering

You must now write a bot client that attempts to win the game in the fewest turns. To do this, you will need to re-implement some subset of the protocol and decode the maze data. Because illegal moves cost you your turn, you must be careful to issue walk or jump commands appropriately based on the decoded maze.

Tips and Caveats

- You will most likely want to use Wireshark or some other network monitoring tool. On some OSes, this may require you to connect remotely, i.e., not to "localhost".
- The maze generation and compression algorithm is based on a depth-first search.
- There exists at least one cheat that will spare you from solving mazes with three jumps. **Do not seek arbitrary code execution on the server.** That is not the sort of cheat I'm talking about. Rather, look for loopholes in the game rules or their implementations.
- Don't spend time discovering security vulnerabilities in the client. There are many. For this lab, focus on the protocol, and the server's implementation of the game rules. If you are uncomfortable running a vulnerable client, then execute the game in a protected environment.

- A nice way to monitor the behavior of your bot is to play against it using the provided client. Avoid having your bot play a game by itself, or else it may complete many games per second. Join the provided client first, then join your bot to the game.
- These development phases need not be considered strictly separate steps. If you find your documentation to be lacking as you work to implement your bot, go back and be sure to record your new experiments in your report. Similarly, if you prefer to begin implementing parts of your bot as you devise your experiments, feel free to do so.

Report

Completion of this lab must be substantiated by a report prepared by the group. It is up to each group to divide its responsibilities fairly. The report should provide sufficient detail that another person could verify your work.

Please include an archive of your source code and resulting binaries in your electronic submission.

Grading

At a minimum, each report should describe the purpose of the lab, the procedure, any assumptions, problems, or hypotheses, and the final results. Mostly, I'll be looking for a proper problem statement, a structured procedure, and for a reasonably correct solution demonstrated by your results.

I will attempt to build your source code and play against your bot. Please include any special instructions for invoking your bot. Part of your grade will depend on its successful execution. It must be able to play the game indefinitely. Each of its moves should bring it closer to the goal by some measure.

Your bot does not need to win every game. If you find yourself stuck trying to optimize the bot's strategy, you've probably missed the cheat. Your bot must not lose a turn, i.e., receive an "illegal move" message from the server. Losing a turn indicates that it cannot decode the maze and/or it does not know the other players' positions. Your bot need not obey every rule, so long as it never loses its turn. I will hold a maze contest for your bots. The winner(s) may receive an extra credit point. The best algorithm, with the cheat, is super simple.

Please only submit one bot. If you submit multiple, I may choose one at random.

This lab is due midnight after 2 weeks.

Rubric

points	Accomplishment
3	Report is well-written (organized, detailed, illustrative)
2	Report provides correct protocol documentation
2	The bot compiles ¹ for a chosen platform
1	The bot plays without crashing
2	The bot never loses its turn because of an illegal move
1	The bot decodes the maze with compression enabled
1	The bot wins (or ties for the win) the maze contest

¹If you choose an interpreted language, these points will be merged with the next entry.